

目录

- 1 实验目的**
- 2 实验平台及环境**
- 3 实验内容**
- 4 实验步骤与结果分析**
 - 4.1 前置步骤
 - 4.2 创建用户
 - 4.3 管理用户
- 5 实验总结与心得体会**

实验四 创建与管理用户

1 实验目的

- 1. 通过实验让学生熟悉并了解 GaussDB 数据库的基本机制与操作。
- 2. 通过用户管理、表管理、数据库对象管理的各种操作，让学生熟悉并了解 DAS 环境下如何使用 GaussDB 数据库管理数据库对象。

2 实验平台及环境

本实验环境为华为云 GaussDB 数据库，具体的配置如表 2-1 所示：

表 2-1 实验环境

配置项	参数
华为云	软件版本：GaussDB v2.0-8.210.0
	设备型号：GaussDB 8 核 64 G
数据库	PostgreSQL

3 实验内容

- 1. 本实验通过用户管理的操作，让学生熟悉并了解 DAS 环境下如何使用 GaussDB 管理用户；
- 2. 本实验通过表管理、数据库对象等管理的操作，让学生熟悉并了解 DAS 环境下如何使用 GaussDB 管理数据库对象。

4 实验步骤与结果分析

为保证报告叙述的连续性，我将“实验步骤”与“实验结果分析”合并为一节，在每个操作步骤之后给出对应的结果和对实验结果的分析。

4.1 前置步骤

1. 登录华为云的 IAM 账户；
2. 登录 GaussDB 数据库实例；
 - (a) 在控制台选择并进入“云数据库 GaussDB”；
 - (b) 选择“gauss-6025-11”实例并登录；

4.2 创建用户

要求：

1. 在 GaussDB 数据库中，通过 `CREATE USER` 创建的用户，默认具有 LOGIN 权限；
2. 通过 `CREATE USER` 创建用户的同时，系统会在执行该命令的数据库中，为该用户创建一个同名的 SCHEMA。其它数据库中，则不自动创建同名的 SCHEMA；
3. 用户可使用 `CREATE SCHEMA` 命令，分别在其他数据库中，为该用户创建同名 SCHEMA；

对应步骤：

1. 选择 SQL 操作，单击 SQL 查询，进入 SQL 查询页面；
2. 选择库名为 `bupt2023212539` 对应的数据库，创建一个新的 schema 为 root，如图 4-1 所示；

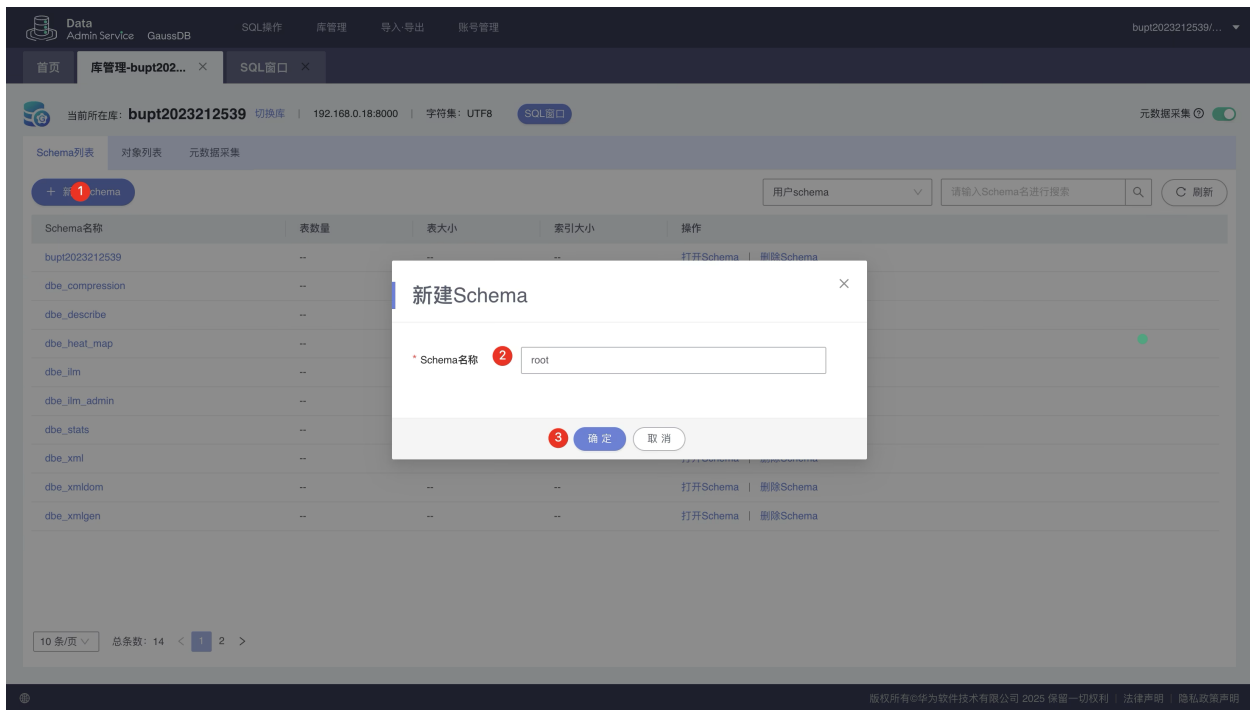


图 4-1 创建名为 root 的 Schema

3. 在 SQL 窗口下打开 root schema，如图 4-2 所示；

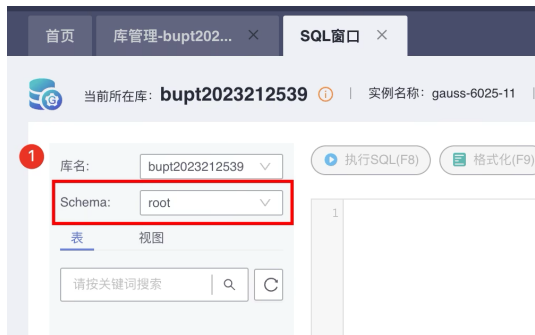


图 4-2 打开 root Schema

4. 创建用户 `user_203212539`，登录密码为 `UserPwd123`，对应操作为在 SQL 窗口输入以下 SQL 命令：

```
1 CREATE USER ems_user <CREATEDB> PASSWORD 'UserPwd123';
```

其中：

- `CREATEDB` 关键字可选，用于创建有“创建数据库”权限的用户。这里我们不添加，而是在后续步骤中进行追加。

注：

- 这里与报告要求的创建 `ems_user` 不同，因为 `ems_user` 已经被其他同学创建并已经在多个数据库里拥有对象或权限。所以，这里创建一个与自己学号相关的用户；
- 由于安全性需要，报告中展示的密码仅作为示例，与实际使用的密码并不一致，后面的操作中密码同样为示例；

实际执行过程如图 4-3 所示：

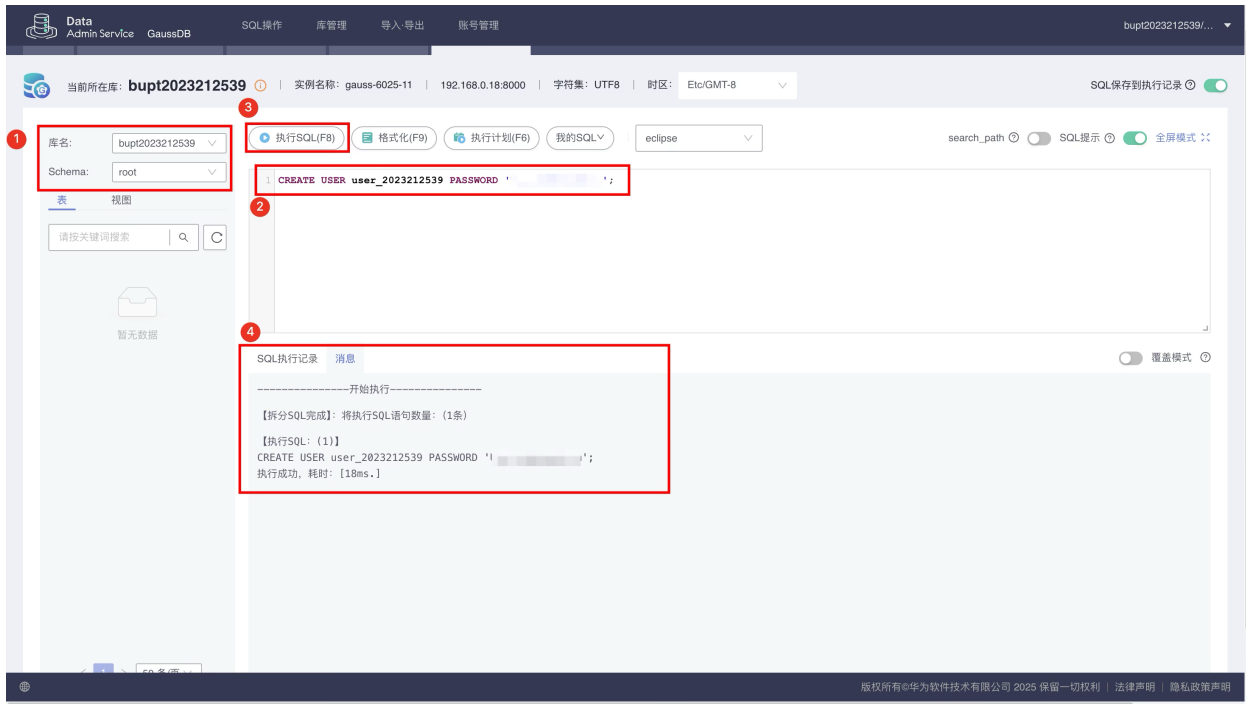


图 4-3 创建用户 `user_2023212539`

执行后，可以发现在执行该命令的数据库，即 `bupt2023212539` 中，系统已经为该用户创建了一个同名的 SCHEMA `user_2023212539`，如图 4-4 所示，与理论一致。

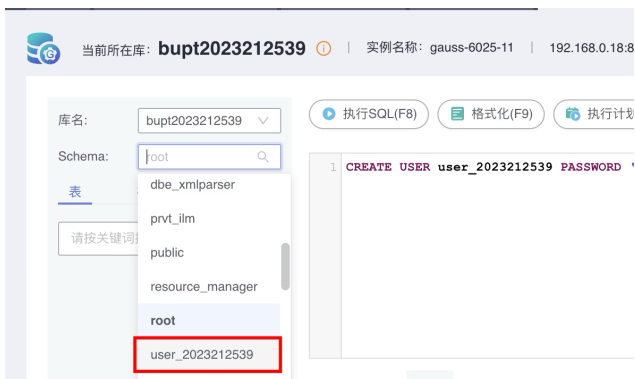


图 4-4 系统在创建用户后自动创建的同名 Schema

4.3 管理用户

1. 选择账号管理，单击角色管理，进入角色管理页面，如图 4-5 所示；

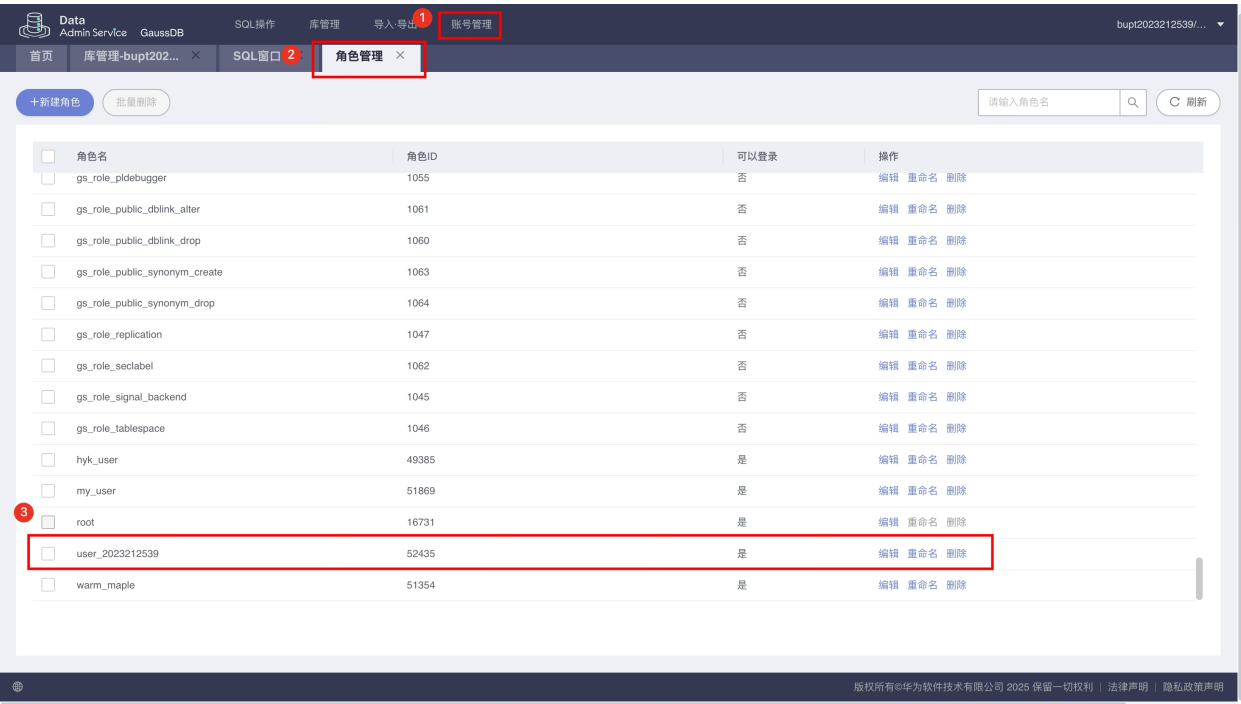


图 4-5 角色管理界面

2. 修改用户登录密码：

- (a) 单击角色名 `user_2023212539`，进入编辑角色页面；
- (b) 在密码框和确认密码框输入新密码，将用户的登录密码修改为 `UserPwd456`，单击保存；
- (c) 显示 SQL 预览，单击确定，修改成功。
- (d) 具体操作如图 4-6 所示；

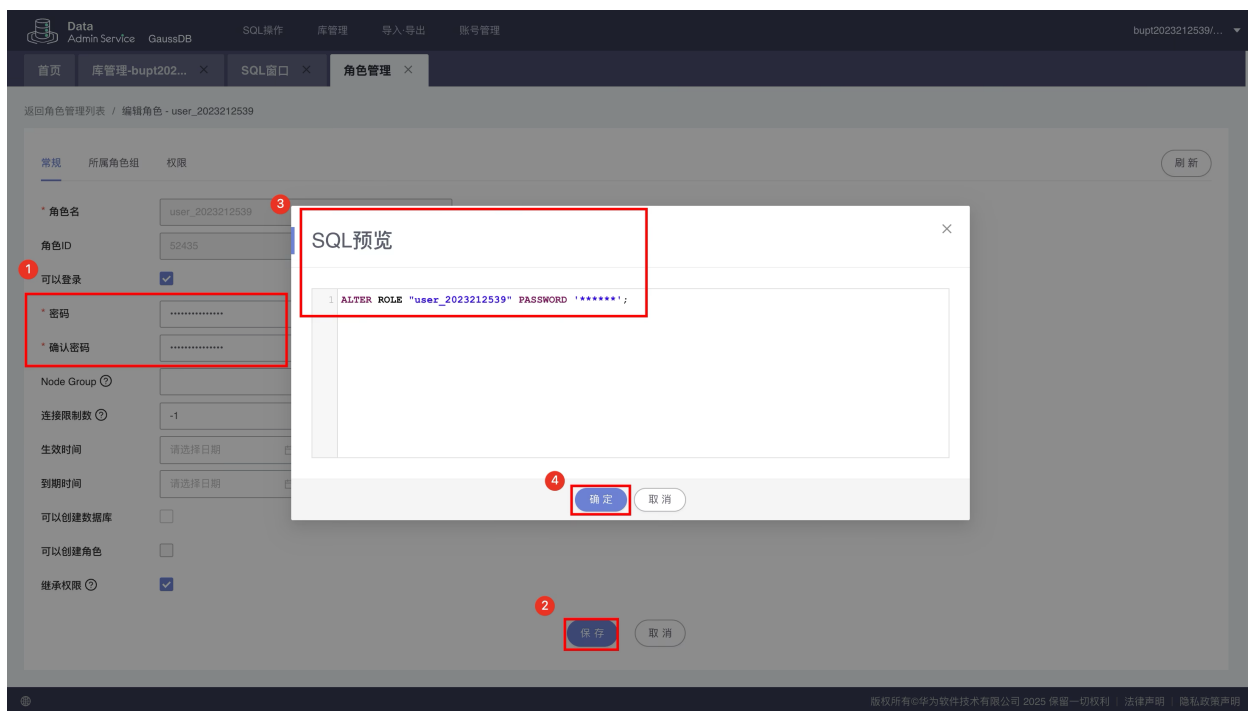


图 4-6 修改 user_2023212539 的密码

从 SQL 预览可以发现，通过 GUI 修改密码的操作等价于以下 SQL 语句：

```
1 ALTER ROLE 'user_2023212539' PASSWORD '*****';  
2 -- 等价于  
3 ALTER USER 'user_2023212539' IDENTIFIED BY '*****';
```

3. 为用户 `user_2023212539` 追加创建数据库的权限，如图 4-7 所示；

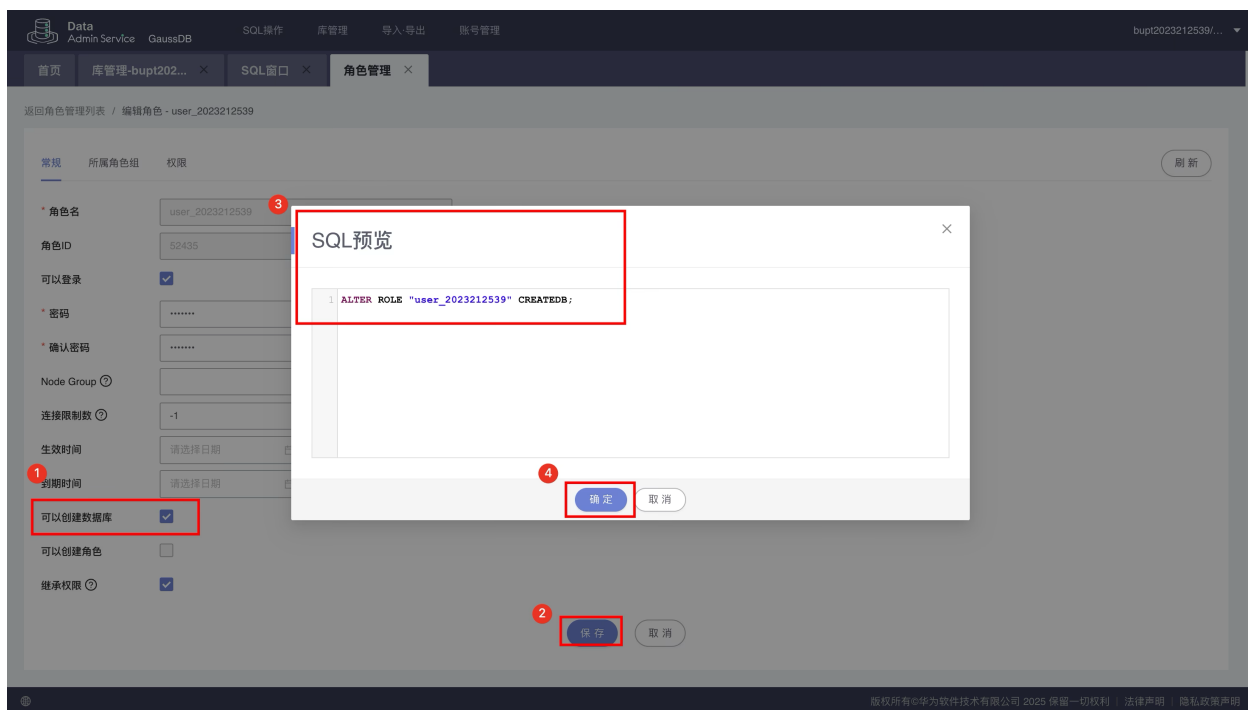


图 4-7 为用户 user_2023212539 追加创建数据库的权限

从 SQL 预览可以发现，通过 GUI 为用户 user_2023212539 追加创建数据库的权限的操作等价于以下 SQL 语句：

```
1 ALTER ROLE 'user_2023212539' CREATEDB;
```

4. 给普通用户 user_2023212539 添加对 products 表的读、写、改权限，其中 products 表只有 product_id 和 product_name 两个字段；
 - (a) （学生用户）创建数据库：选择 SQL 操作，单击 SQL 查询；进入 SQL 查询页面，库名选择 bupt2023212539，Schema 选择 root，如图 4-8 所示；
 - (b) （学生用户）新建 products 表并插入数据，对应的 SQL 语句见下文，执行过程如图 4-9 和 4-10 所示；
 - (c) （学生用户）授予普通用户 user_2023212539 连接当前数据库的权限，对应的 SQL 语句见下文，执行过程如图 4-11 所示；

注：这一步不同于实验指导手册，虽然普通用户被创建时默认具有 LOGIN，即登录权限，但实际在实验过程中发现，还需要额外授予 CONNECT 权限在能正确登录个人数据库。

- (d) （学生用户）授予普通用户 user_2023212539 使用 root schema 的权限，对应的 SQL 语句见下文，执行过程如图 4-12 所示；
 - (e) （学生用户）给普通用户 user_2023212539 添加对 products 表的读、写、改权限，对应的 SQL 语句见下文，执行过程如图 4-13 所示；

注：上文以及之后出现的在具体操作前的“（学生用户）”或“（普通用户）”表示分别登录在 bupt2023212539 和 user_2023212539 用户下执行操作。

上文涉及的有关 SQL 语句如下：

```
1 -- 创建 products 表
2 CREATE TABLE products (
3     product_id SERIAL PRIMARY KEY,
4     product_name VARCHAR(160) NOT NULL
5 );
6
7 -- 插入数据
8 INSERT INTO products (product_name) VALUES
9 ('Apple iPhone 16 Pro'),
10 ('Samsung Galaxy S24'),
11 ('Dell XPS 13 Laptop');
```

```

12 ('Sony WH-1000XM5 Headphones'),
13 ('Nintendo Switch OLED'),
14 ('Logitech MX Master 3S Mouse'),
15 ('Kindle Paperwhite 11th Gen'),
16 ('Xiaomi Mi Band 8'),
17 ('Apple AirPods Pro 2'),
18 ('Huawei MatePad 11 Tablet');
19
20 -- 授予权限
21 GRANT CONNECT ON DATABASE bupt2023212539 TO user_2023212539;
22 GRANT USAGE ON SCHEMA root TO user_2023212539;
23 GRANT SELECT, INSERT, UPDATE ON TABLE products TO user_2023212539;

```

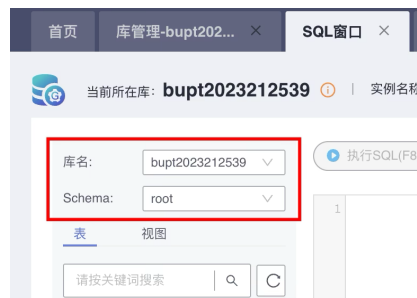


图 4-8 进入自己的数据库并选择 root Schema

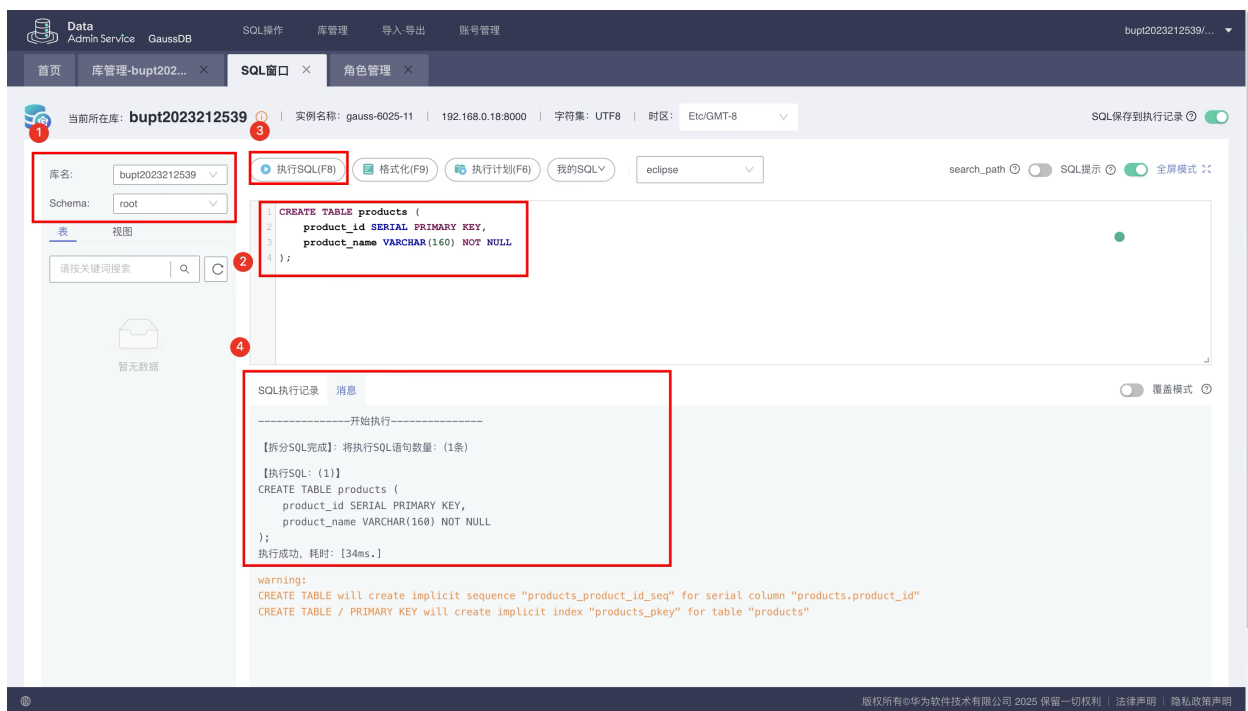


图 4-9 新建 products 表

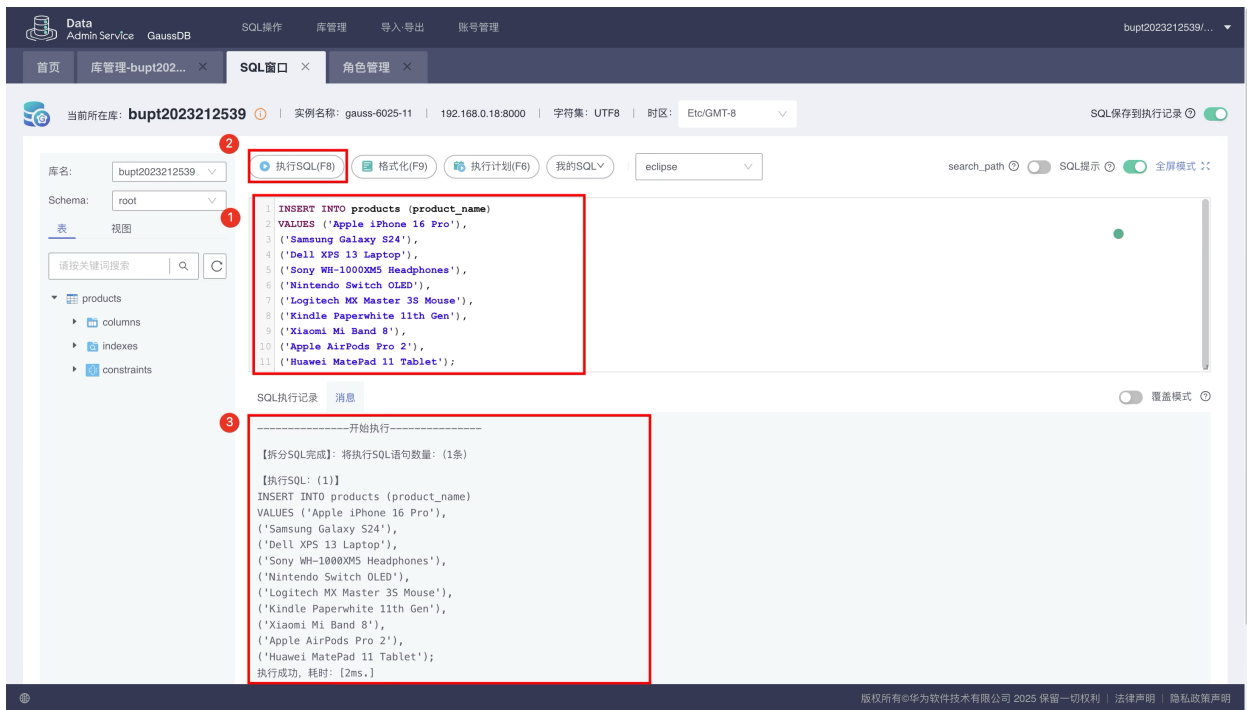


图 4-10 在 products 表中插入数据

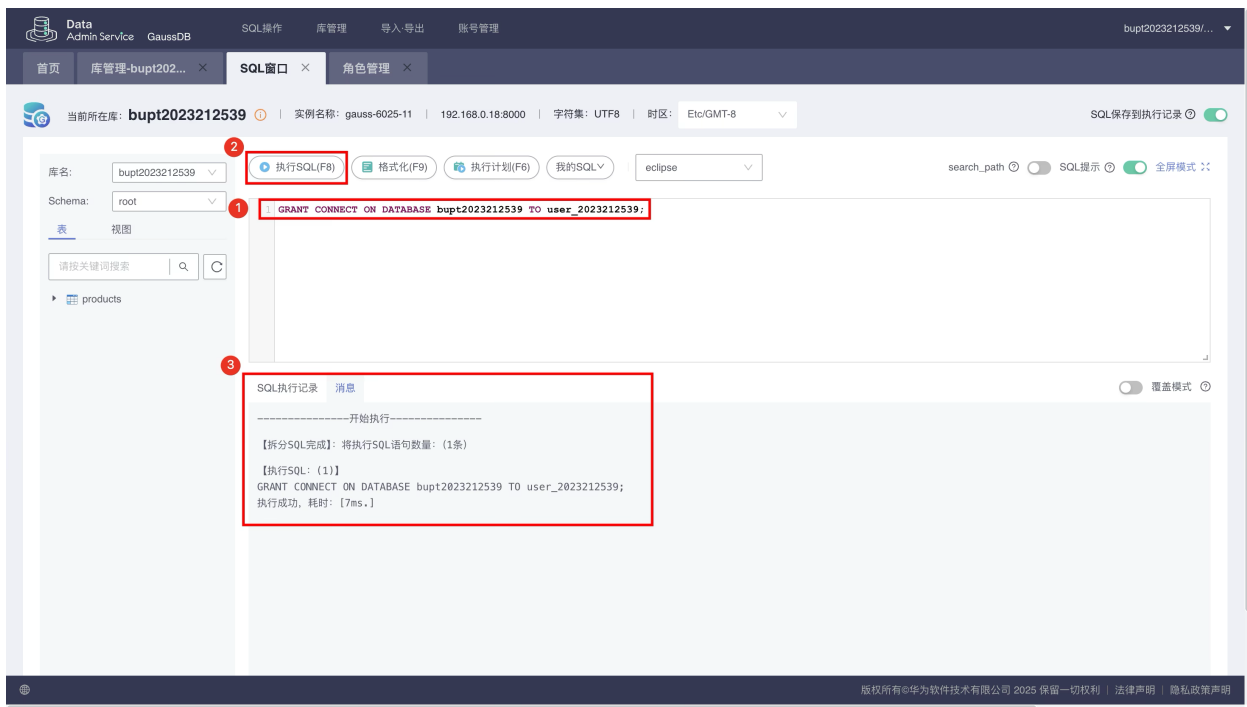


图 4-11 授予普通用户连接个人数据库的权限

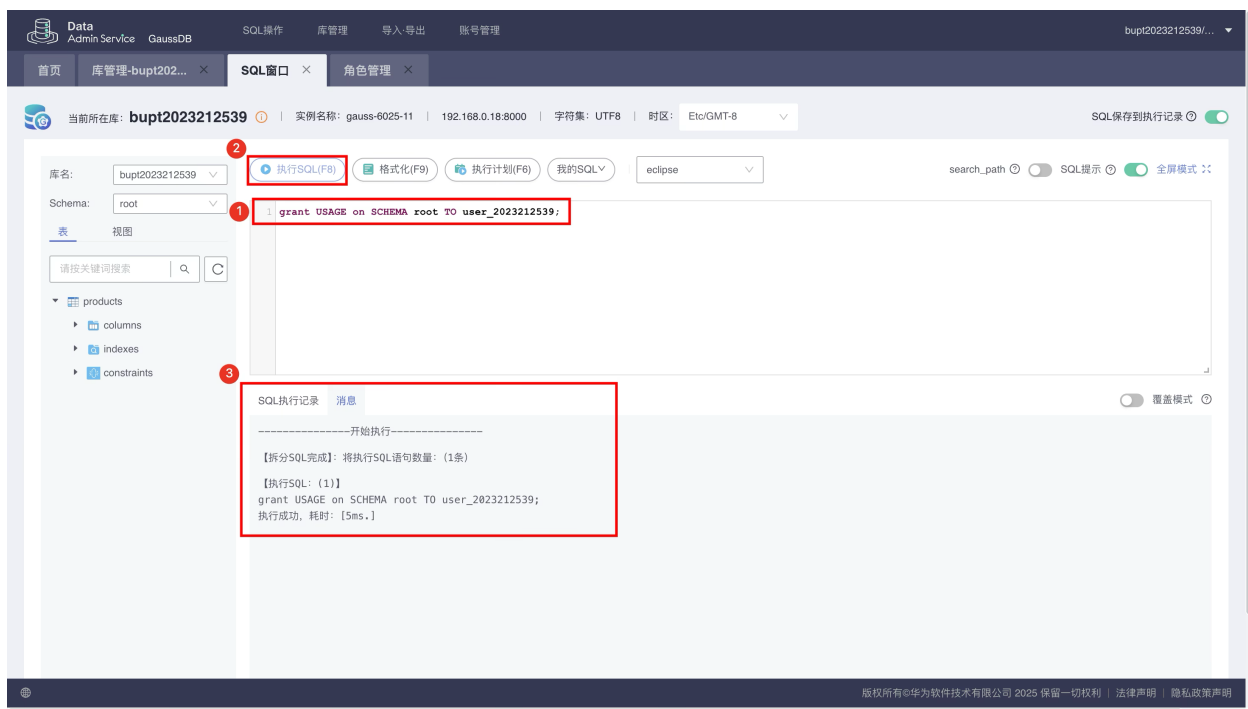


图 4-12 授予普通用户使用 root schema 的权限

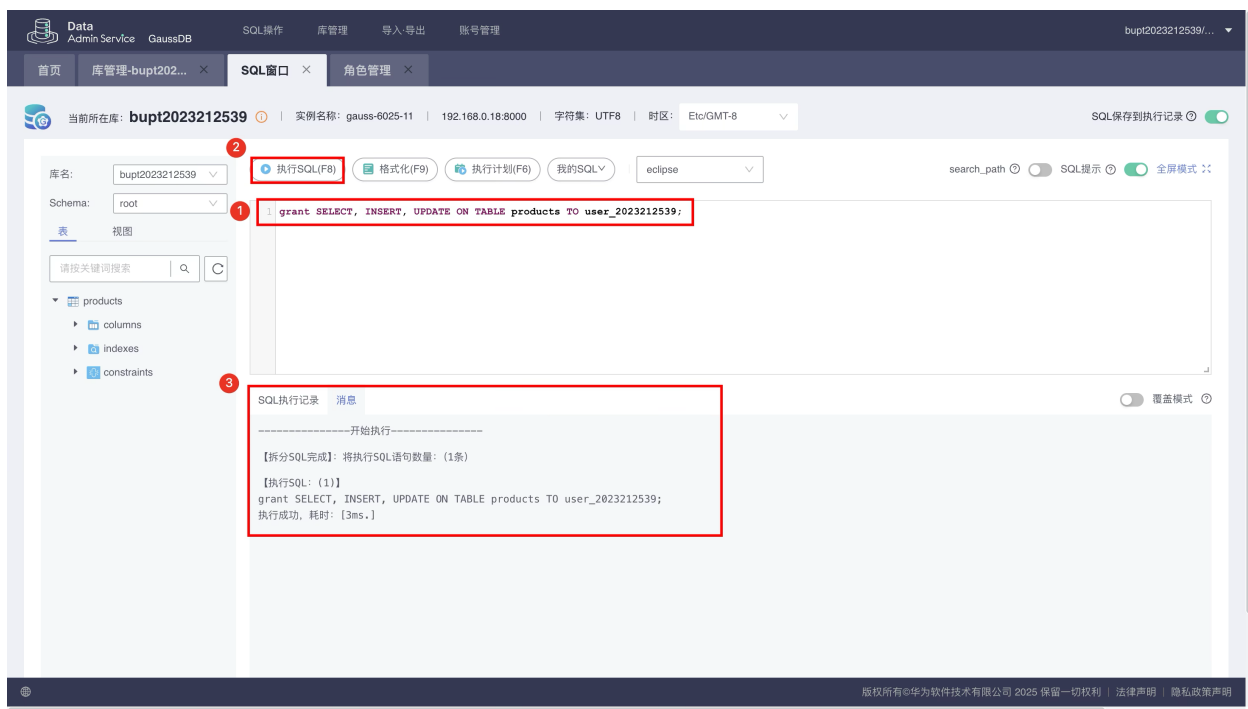


图 4-13 给普通用户添加对 products 表的读、写、改权限

5. 验证账户权限：

- (a) （学生用户）单击右上角账户名，选择切换连接，自定义登陆切换 `user_2023212539` 用户，执行过程如图 4-14 和图 4-15 所示；

注：这里数据库名称可以选择 `postgres`，这样无需执行我上文中提到的额外赋予 `CONNECT` 权限的操作。

- (b) （普通用户）查询 `products` 表内容，对应的 SQL 语句见下文，执行过程如图 4-16 所示，SQL 执行后正确得到插入的 10 条数据，说明普通用户 `user_2023212539` 的访问 root schema 权限和对 `products` 表的读权限配置正确；
- (c) （普通用户）尝试删除 `products` 表中编号为 1 的商品，对应的 SQL 语句见下文，执行过程如图 4-17 所示，SQL 执行后报错“**ERROR: Permission denied for relation products.**”，与理论一致，因为我在前面并没有授予普通用户对 `products` 表的 `DELETE` 权限；
- (d) （普通用户）尝试修改 `products` 表中编号为 2 的商品的 `product_name` 为“Samsung Galaxy S25”，对应的 SQL 语句见下文，执行过程如图 4-18 所示；
- (e) （普通用户）查看更新后的 `products` 表结果，对应的 SQL 语句见下文，执行过程如图 4-19 所示，SQL 执行后，编号为 2 的商品名称从“Samsung Galaxy S24”正确更新为“Samsung Galaxy S25”，说明普通用户 `user_2023212539` 的对 `products` 表的更新权限配置正确；

上文涉及的有关 SQL 语句如下：

```
1  -- 查询表内容
2  SELECT * FROM products
3
4  -- 尝试执行删除操作
5  DELETE FROM products WHERE product_id = 1
6
7  -- 更新商品名称
8  UPDATE products SET product_name = 'Samsung Galaxy S25' WHERE product_id = 2;
9
10 -- 查看更新后结果
11 SELECT * FROM products
```



图 4-14 切换登录为普通用户

×

1

已有连接登录 自定义登录

* 数据库:

* 用户名:

* 密码:

☒ 记住密码

同意用户名及密码记录到DAS系统中，如不再需要，可以在数据库登录列表中删除。
若此项不开启，DAS只能实时去数据库查询这些结构定义数据，对您的数据库实时性能有一定的影响。

☒ 命令执行记录 ⓘ

开启此项后，您可以在DAS中，方便的查看到您的命令窗口执行历史记录，并且可以直接再次执行，无需重复输入。

2

登录

列表中的数据库账户信息来源于您之前记录在DAS服务中的信息，如不再需要，您可以在数据库登录列表修改或删除他们。

图 4-15 登录普通用户

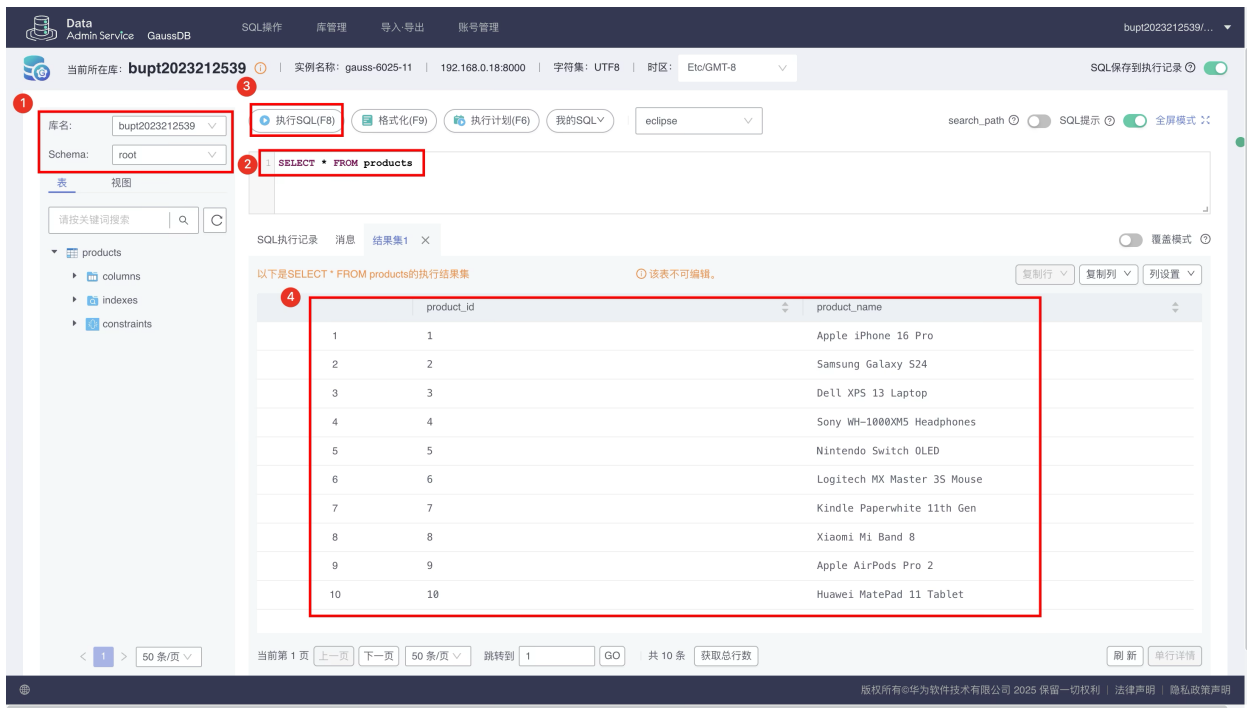


图 4-16 查看 products 表内容

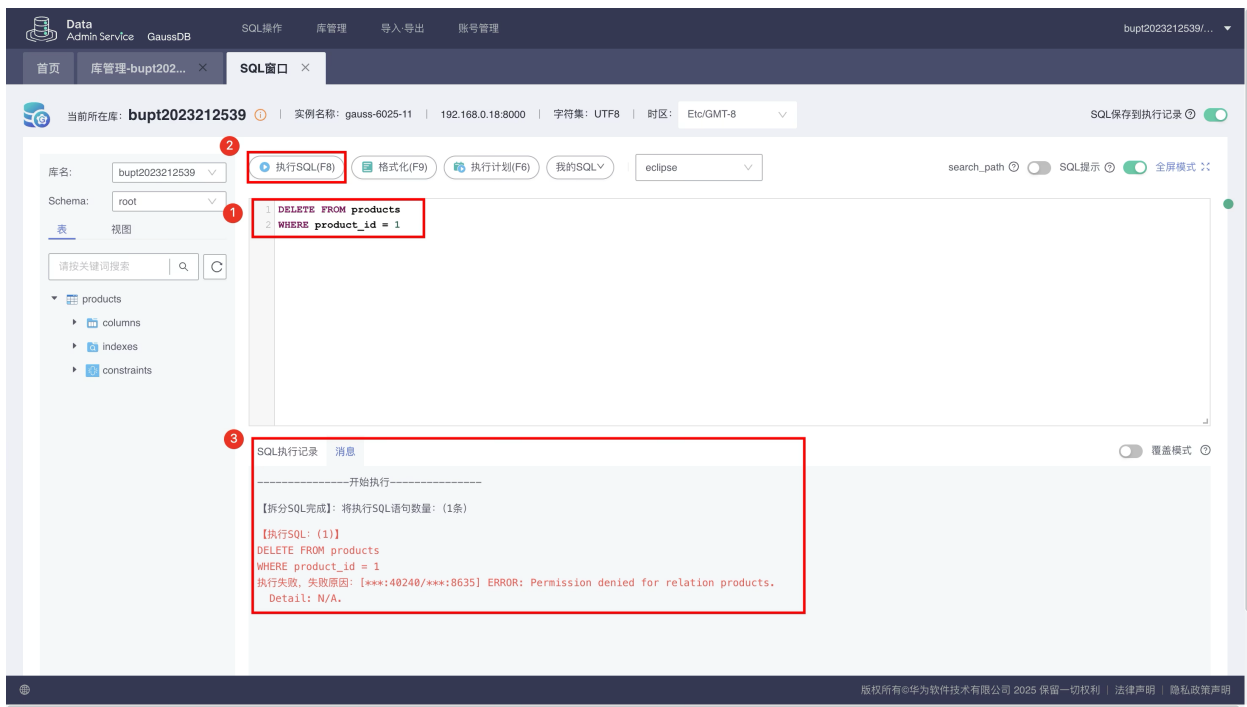


图 4-17 尝试删除 products 表中数据

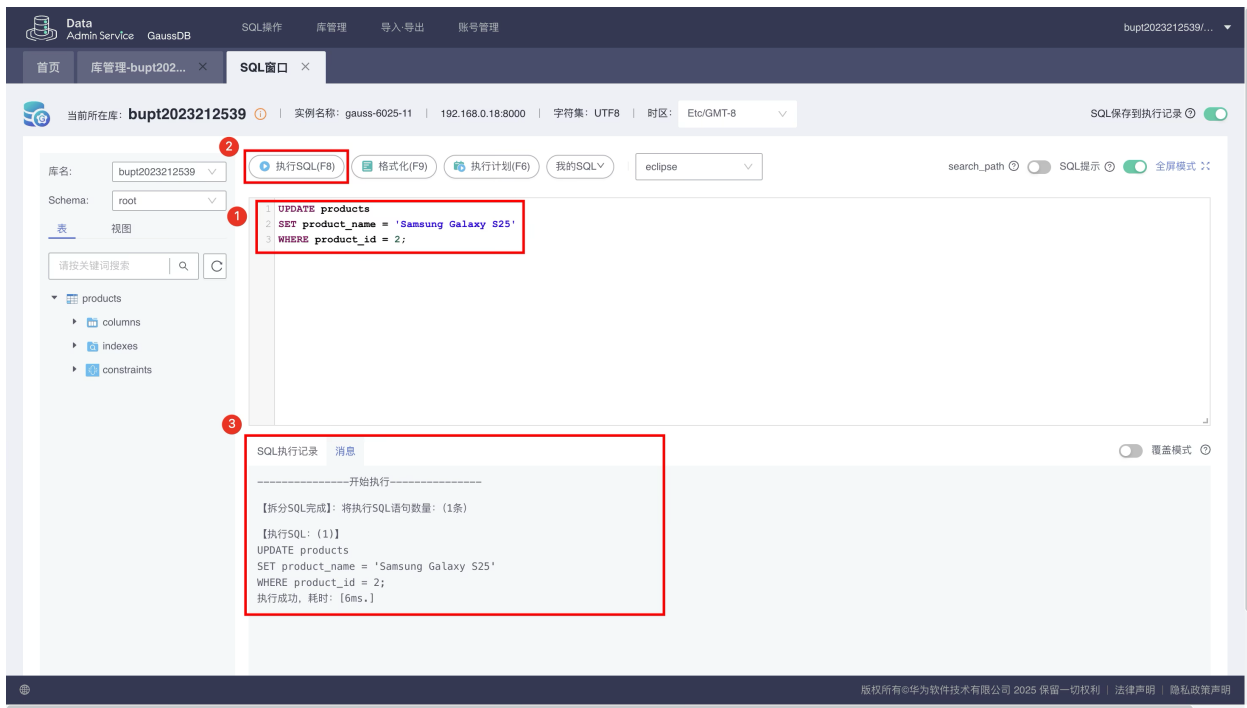


图 4-18 更新 products 表中数据

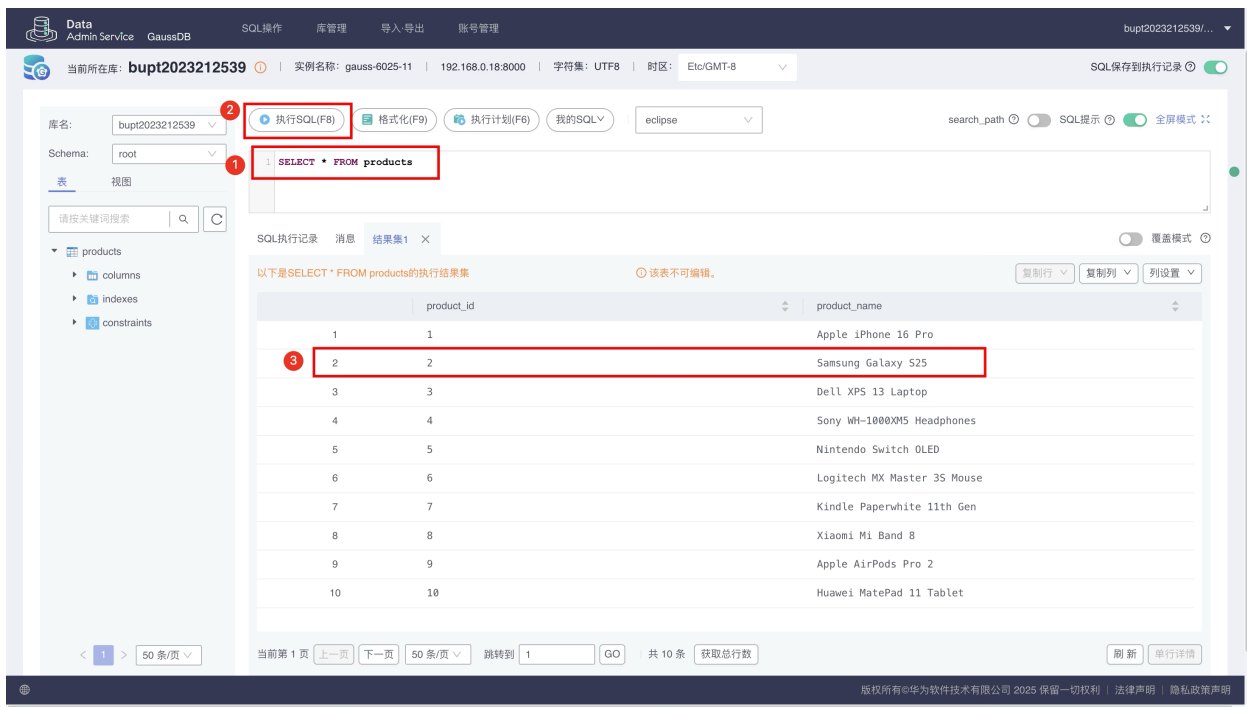


图 4-19 更新后 products 表内容

6. 回收用户 `user_2023212539` 对 `products` 表的写权限和修改权限，只保留查询（`SELECT`）的权限。

- (a) （普通用户）单击右上角账户名，选择切换连接，在已有连接登录里切换回学生账户，选择 SQL 操作，单击 SQL 查询，进入 SQL 查询页面，库名选择 `bupt2023212539`，Schema 选择 root，执行过程如图 4-20 和图 4-21 所示；
- (b) （学生用户）回收用户 `user_2023212539` 对 `products` 表的写权限和修改权限，对应的 SQL 语句见下文，执行过程如图 4-22 所示；

上文涉及的有关 SQL 语句如下：

1	-- 收回写权限和修改权限
2	REVOKE INSERT, UPDATE ON TABLE products FROM user_2023212539;



图 4-20 切换登录为学生用户

图 4-21 登录学生用户

图 4-22 收回普通用户对 products 表的写、修改权限

7. 验证权限是否正确收回：
- (a) （学生用户）单击右上角账户名，选择切换连接，自定义登陆切换 `user_2023212539` 用户，该操作与之前一致，如图 4-14 和 4-15；
- (b) （普通用户）尝试给 `products` 表添加一条商品名为“Sony PlayStation 5 Pro”的商品数据，对应的 SQL 语句见下文，执行过程如图 4-23 所示，SQL 执行后报错“**ERROR: Permission denied for relation products.**”，与理论一致，普通用户已经无法写入表内容，说明普通用户对 `products` 表的写权限（`INSERT`）已经正确收回；

- (c) （普通用户）尝试给 `products` 表修改商品编号为 2 的商品名称为“Samsung Galaxy S26”，对应的 SQL 语句见下文，执行过程如图 4-24 所示，SQL 执行后报错“**ERROR: Permission denied for relation products.**”，与理论一致，普通用户已经无法更新表内容，说明普通用户对 `products` 表的更新权限（`UPDATE`）已经正确收回；
- (d) 查询 `products` 的所有商品，对应的 SQL 语句见下文，执行过程如图 4-25 所示，SQL 执行后正确得到商品数据，说明普通用户 `user_2023212539` 对 `products` 表仍有读权限；

上文涉及的有关 SQL 语句如下：

```
1  -- 插入数据
2  INSERT INTO products (product_id, product_name) VALUES (11, 'Sony PlayStation
3  5 Pro');
4  -- 更新商品名称
5  UPDATE products SET product_name = 'Samsung Galaxy S26' WHERE product_id = 2;
6
7  -- 查询表内容
8  SELECT * FROM products
```

图 4-23 尝试写入 products 表

图 4-24 尝试更新 products 表中数据

图 4-25 查看 products 表中数据

5 实验总结与心得体会

本次实验深入实践了 GaussDB 数据库中的用户创建与权限管理操作，让我对数据库安全和对象管理机制有了更直观和深刻的理解。通过实际操作 `CREATE USER`、`ALTER ROLE`、`GRANT` 和 `REVOKE` 等 SQL 命令，我熟练掌握了如何在一个 DAS 环境下管理数据库用户及其对特定数据库对象的权限。

实验中，我遇到了一个关键性的细节问题：虽然创建新用户 `user_2023212539` 时默认具有 `LOGIN` 权限，但在实际尝试登录并连接到特定的个人数据库 `bupt2023212539` 时，却发现无法直接访问。经过查阅和尝试，我发现还需要额外使用 `GRANT CONNECT ON DATABASE bupt2023212539 TO user_2023212539` 命令来显式授予用户连接到该数据库的权限，这补充了我对 PostgreSQL 权限模型的认识，即登录（`LOGIN`）仅是允许用户通过身份验证，而连接（`CONNECT`）则是允许用户访问特定的数据库实例。

此外，为了让普通用户能够访问学生用户创建的 `products` 表，还需要授予 `GRANT USAGE ON SCHEMA root` 的权限，这体现了数据库对象（如表）访问的层级性，必须首先拥有对包含该对象的 Schema 的使用权。

在权限验证部分，通过授予普通用户 `SELECT`、`INSERT`、`UPDATE` 权限，并刻意不授予 `DELETE` 权限，我成功验证了权限的粒度控制。普通用户可以查询、插入和修改数据，但在尝试执行 `DELETE` 操作时立刻收到了“ERROR: Permission denied for relation products.”的错误，这直观地展示了权限模型的有效性和精细性。随后，通过 `REVOKE INSERT, UPDATE` 操作并再次验证，我确认了权限回收的即时性和准确性。

综上，本次实验让我掌握了用户创建、管理的基本操作，让我熟悉了用户和权限管理的相关知识，也了解了不同权限的区别。

一些对实验的建议：

对于本次实验，我认为可以适当增加别的权限操作的实践，例如 `DROP` 权限、对视图（`VIEW`）或函数（`FUNCTION`）的权限管理，以进一步拓展我们对数据库安全模型的理解。